MODUL 12 · SUPABASE BAAS

Backend as a Service dengan Supabase

Mengenal Supabase sebagai backend siap pakai berbasis PostgreSQL. Mahasiswa membuat project, menyimpan kunci dengan aman, membuat tabel lewat SQL Editor, mengamankan data dengan Row Level Security, lalu melakukan query CRUD, autentikasi, Realtime, dan Storage dari Flutter.

Durasi Pertemuan 12 (150 Menit) + Belajar Mandiri

Prasyarat Modul 08: State Management Provider
Modul 09: REST API & JSON
Modul 11: Hive
Mata kuliah Basis Data (SQL dasar)

Capaian Pembelajaran

- Menjelaskan komponen Supabase (Database, Auth, RLS, Realtime, Storage) dan membandingkannya dengan Firebase.
- Membuat project Supabase, menemukan Project URL dan publishable key, serta membedakannya dengan secret key.
- Mengonfigurasi supabase_flutter dengan Supabase.initialize dan --dart-define-from-file tanpa membocorkan kunci ke repository.
- Membuat tabel dengan SQL (tipe data, default, constraint, uuid, foreign key ke auth.users).
- Menulis policy Row Level Security memakai auth.uid() dan menjelaskan perbedaan using dan with check.
- Melakukan select (filter, order, limit, range), insert, update, delete, upsert, single dan maybeSingle dari Flutter.
- Menangani PostgrestException, AuthException, dan kegagalan jaringan dengan pesan yang ramah pengguna.
- Menggunakan signUp, signInWithPassword, signOut, currentSession, dan onAuthStateChange.

Daftar Isi Modul

1. Apa itu Supabase? (vs Firebase)
2. Membuat Project & Menemukan API Key
3. Instalasi supabase_flutter & Supabase.initialize
4. Keamanan Kunci: env.json & --dart-define

5. Membuat Tabel dengan SQL Editor
6. Row Level Security (RLS) & Policy
7. Query Dasar: Select, Insert, Update, Delete, Upsert
8. Menangani Error Supabase
9. Dasar Supabase Auth
10. Sekilas Realtime & Storage
11. Tugas Praktikum
12. Kuis Pemahaman & Kunci Jawaban

1. Apa itu Supabase? (vs Firebase)

Sampai Modul 11, semua data aplikasi kita tersimpan di HP (SharedPreferences, SQLite, Hive). Data itu hilang kalau aplikasi di-uninstall dan tidak bisa dibuka dari HP lain. Untuk aplikasi sungguhan kita butuh **backend**: server yang menyimpan data di cloud dan mengatur siapa boleh mengakses apa.

Apa itu Backend as a Service (BaaS)?

Membuat backend sendiri berarti menyewa server, memasang database, menulis REST API (seperti yang kita konsumsi di Modul 09), membuat sistem login, mengurus keamanan, dan seterusnya. **BaaS** menyediakan semua itu dalam bentuk siap pakai. Kita cukup membuat tabel lewat dashboard, lalu memanggilnya dari Flutter dengan library resmi.

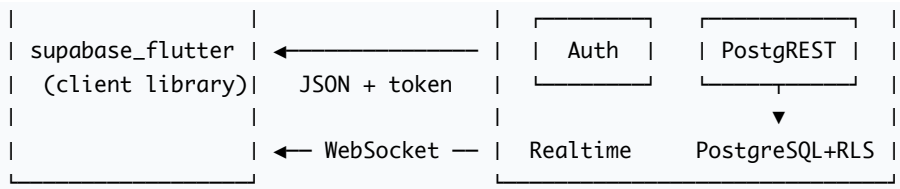
Supabase adalah BaaS open source yang dibangun di atas database **PostgreSQL**. Karena memakai PostgreSQL, semua yang Anda pelajari di mata kuliah Basis Data (tabel, relasi, *primary key*, *foreign key*, SQL) langsung terpakai.

Fitur utama Supabase

Fitur	Fungsi	Dipakai di modul
Database (Postgres)	Tabel relasional sungguhan. Bisa diakses dari Flutter tanpa menulis REST API sendiri, karena Supabase membuat API-nya otomatis (PostgREST).	12, 14
Auth	Register, login email/password, magic link, Google, dan lain-lain. Sesi login disimpan dan diperbarui otomatis.	12, 13
Row Level Security (RLS)	Aturan keamanan per baris di database, misalnya "user hanya boleh melihat catatannya sendiri".	12, 13, 14
Realtime	Aplikasi menerima perubahan data secara langsung (seperti chat) tanpa perlu refresh.	12
Storage	Menyimpan file: foto profil, PDF tugas, dan sejenisnya.	12 (singkat)
Edge Functions	Menjalankan kode server (TypeScript) untuk logika khusus.	Di luar cakupan

Bagaimana Flutter berbicara dengan Supabase?





Kita tidak menulis URL endpoint satu per satu seperti di Modul 09. Cukup panggil `supabase.from('tabel').select()`. Library-lah yang mengubahnya menjadi request HTTP, menyertakan token login, lalu mengembalikan JSON yang sudah di-decode.

Supabase vs Firebase

Firebase (Google) adalah BaaS paling populer di dunia Flutter. Keduanya bagus. Perbedaan utamanya:

Aspek	Supabase	Firebase
Jenis database	Relasional (PostgreSQL), memakai SQL	NoSQL dokumen (Cloud Firestore)
Relasi antar data	Foreign key dan JOIN bawaan	Denormalisasi manual
Aturan keamanan	Row Level Security (policy SQL)	Security Rules (bahasa khusus)
Open source / self-host	Ya, bisa dipasang di server sendiri	Tidak
Setup di Flutter	Cukup URL + key, tanpa file konfigurasi native	Perlu FlutterFire CLI dan file konfigurasi per platform
Paket gratis	Ada (project bisa di-*pause* otomatis bila lama tidak aktif)	Ada (Spark plan)
Cocok untuk	Data terstruktur, mahasiswa yang sudah belajar SQL	Data fleksibel, ekosistem Google

> ℹ️ **Kenapa kita memilih Supabase?**
>
> Kurikulum Informatika sudah membekali Anda dengan SQL dan desain basis data relasional. Dengan Supabase, ilmu itu langsung terpakai, dan konsep keamanan per baris (RLS) adalah praktik yang juga dipakai di dunia industri.

Peta modul berikutnya

- **Modul 12 (ini):** dasar Supabase, yaitu membuat project, tabel, RLS, query CRUD, Auth, dan sekilas Realtime & Storage.
- **Modul 13:** Proyek "Catatan Kuliah" bagian 1, berisi arsitektur aplikasi serta login & register lengkap.
- **Modul 14:** Proyek bagian 2, yaitu CRUD catatan dengan Supabase plus cache offline memakai Hive.

2. Membuat Project & Menemukan API Key

Sebelum menulis kode Flutter, kita siapkan dulu "server"-nya, yaitu sebuah *project* di Supabase. Semua langkah di bawah gratis dan tidak memerlukan kartu kredit.

Langkah 1: Membuat akun

1. Buka supabase.com, klik **Start your project**.
2. Pilih **Continue with GitHub** (paling cepat) atau daftar dengan email.
3. Saat pertama kali masuk, Anda diminta membuat **Organization**. Isi nama (misalnya `praktikum-mobile`), pilih tipe *Personal* dan plan *Free*.

Langkah 2: Membuat project baru

1. Di dashboard klik **New project**.
2. **Project name**: `catatan-kuliah`.
3. **Database Password**: klik *Generate a password*, lalu **simpan** di tempat aman. Password ini adalah password database Postgres (dipakai untuk koneksi langsung ke database), *bukan* password login aplikasi.
4. **Region**: pilih yang paling dekat dengan pengguna, yaitu **Southeast Asia (Singapore)**, agar latensi kecil.
5. Klik **Create new project** dan tunggu 1–2 menit sampai status project siap.

Langkah 3: Menemukan Project URL dan API key

Aplikasi Flutter hanya butuh **dua nilai** untuk terhubung:

1. Klik tombol **Connect** di bagian atas dashboard project, atau buka **Project Settings** (ikon roda gigi) → **API Keys**.
2. **Project URL** berbentuk `https://<project-ref>.supabase.co`. `project-ref` adalah 20 huruf acak, sama dengan yang terlihat di address bar dashboard.
3. **Publishable key** berbentuk `sb_publishable_...`. Inilah kunci yang dipasang di aplikasi.

Jenis kunci	Bentuk	Boleh di aplikasi Flutter?	Keterangan
Publishable key	`sb_publishable_...`	✅ Ya	Kunci "publik". Aman ada di aplikasi *asalkan RLS aktif*. Pengganti anon key.
Anon key (legacy)	JWT panjang `eyJhbGciOi...`	✅ Ya (cara lama)	Ada di tab *Legacy API Keys*. Fungsinya sama dengan publishable key.
Secret key	`sb_secret_...`	❌ TIDAK PERNAH	Melewati semua RLS (akses penuh). Hanya untuk server/backend.
service_role (legacy)	JWT panjang	❌ TIDAK PERNAH	Versi lama dari secret key, sama berbahayanya.

ℹ️ **Publishable key vs anon key**

Supabase memperkenalkan format kunci baru `sb_publishable_...` dan `sb_secret_...`. Package `supabase_flutter` versi terbaru memakai parameter `publishableKey` di `Supabase.initialize()`. Parameter lama `anonKey` masih ada tetapi sudah *deprecated*. Jika Anda menemukan tutorial lama yang menulis `anonKey: 'eyJ...'`, cara itu masih berjalan, tetapi untuk project baru gunakan `publishableKey`.

💡 **Tampilan dashboard berubah?**

Supabase cukup sering memperbarui tampilan dashboard. Jika menu di atas tidak persis sama, cari kata kunci **API Keys** atau tombol **Connect**. Nilai yang dicari tetap sama, yaitu Project URL dan publishable key.

Langkah 4: Mengenal menu dashboard yang akan sering dipakai

Menu	Kegunaan
Table Editor	Melihat dan mengedit isi tabel seperti spreadsheet.
SQL Editor	Menjalankan perintah SQL: membuat tabel, policy, trigger. Kita akan banyak memakai menu ini.
Authentication	Daftar user yang sudah register, pengaturan provider login (Email), dan template email.
Storage	Membuat *bucket* untuk file.
Database → Policies	Melihat policy RLS setiap tabel (juga tersedia lewat Authentication → Policies).
Logs	Melihat request yang masuk dan error, berguna saat debugging.

> ⚠️ **Project gratis bisa di-pause**
>
> Project pada plan Free akan di-*pause* otomatis bila tidak ada aktivitas selama beberapa waktu. Jika aplikasi tiba-tiba tidak bisa terhubung menjelang praktikum, buka dashboard lalu klik **Restore project**.

3. Instalasi supabase_flutter & Supabase.initialize

Supabase menyediakan package resmi `supabase_flutter`. Di dalamnya sudah ada semua yang dibutuhkan: client database (PostgREST), Auth, Realtime, dan Storage, termasuk penyimpanan sesi login otomatis di perangkat.

Langkah 1: Tambahkan package

```
flutter pub add supabase_flutter
```

Perintah ini menambahkan baris `supabase_flutter: ^x.y.z` di `pubspec.yaml` dengan versi terbaru yang kompatibel, lalu menjalankan `flutter pub get`.

Langkah 2: Inisialisasi di `main()`

Supabase harus di-*initialize* **satu kali**, sebelum `runApp()`. Contoh paling sederhana (nilai URL dan key masih ditulis langsung; di section berikutnya kita pindahkan ke file konfigurasi):

```dart
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

Future<void> main() async {
  // 1. Wajib: memastikan binding Flutter siap sebelum kode async + plugin.
  WidgetsFlutterBinding.ensureInitialized();

  // 2. Hubungkan aplikasi ke project Supabase (cukup SEKALI).
  await Supabase.initialize(
    url: 'https://abcdefghijklm.supabase.co',
    publishableKey: 'sb_publishable_xxxxxxxxxxxxxxxxxxxx',
  );

  runApp(const MaterialApp(home: Scaffold()));
}

// 3. Ambil client di mana saja setelah initialize.
final supabase = Supabase.instance.client;
```

Penjelasan baris penting:

- `main()` diubah menjadi `async` dan mengembalikan `Future<void>` karena kita perlu `await` sebelum `runApp()`.
- `WidgetsFlutterBinding.ensureInitialized()` wajib dipanggil sebelum memakai plugin (di sini untuk penyimpanan sesi) di dalam `main()`. Lupa baris ini akan memunculkan error *Binding has not yet been initialized*.
- `Supabase.initialize(url:, publishableKey:)` membuat client sekaligus memulihkan sesi login yang tersimpan dari pembukaan aplikasi sebelumnya.
- `Supabase.instance.client` adalah objek `SupabaseClient` yang kita pakai untuk semua operasi. Memanggilnya *sebelum* initialize selesai akan error.

Langkah 3: Izin internet di Android

Proyek Flutter baru hanya memberi izin internet untuk mode *debug*. Agar APK *release* juga bisa mengakses Supabase, tambahkan izin di manifest utama:

`android/app/src/main/AndroidManifest.xml`

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <!-- Tambahkan baris ini, di atas tag <application> -->
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:label="catatan_kuliah"
        ...
```

> 💡 **Platform lain**
>
> iOS tidak membutuhkan izin tambahan. Untuk macOS, aktifkan `com.apple.security.network.client` di file `macos/Runner/DebugProfile.entitlements` dan `Release.entitlements`. Flutter Web langsung bisa dipakai.

Langkah 4: Uji koneksi

Jalankan aplikasi. Jika tidak ada error merah di konsol, inisialisasi berhasil. Perlu diingat, `initialize` tidak mengirim query ke database, jadi URL atau key yang salah baru ketahuan saat query pertama dijalankan (akan kita coba di section Query).

> ⚠️ **Kesalahan umum**
>
> - **"You must initialize the supabase instance before calling Supabase.instance"**: `Supabase.instance.client` dipanggil sebelum `await Supabase.initialize(...)`, misalnya di variabel global yang dibaca terlalu awal atau karena lupa `await`.
> - **Memanggil initialize di dalam `build()`**: initialize cukup sekali di `main()`.
> - **URL diakhiri `/rest/v1`**: cukup tulis URL dasarnya, `https://xxxx.supabase.co`.

4. Keamanan Kunci: env.json & --dart-define

Menulis URL dan key langsung di kode (*hard-code*) memang praktis, tetapi berisiko: begitu kode di-*push* ke GitHub publik, siapa pun bisa melihat dan memakai project Anda. Di section ini kita belajar memisahkan konfigurasi dari kode.

Aturan emas

1. **Jangan pernah** memasukkan *secret key* (`sb_secret_...`) atau *service_role* ke aplikasi Flutter. Semua isi APK bisa dibongkar (*decompile*), dan kunci itu melewati seluruh RLS, sehingga pemegangnya bisa membaca dan menghapus data semua user.
2. Publishable key **memang boleh** ada di aplikasi. Yang melindungi data adalah **RLS**, bukan kerahasiaan publishable key.
3. Meskipun publishable key tidak rahasia, tetap simpan di file konfigurasi terpisah yang **tidak di-commit**. Dengan begitu setiap anggota tim bisa memakai project masing-masing tanpa mengubah kode, dan repo tidak menjadi sasaran penyalahgunaan kuota.

Cara 1: `--dart-define`

Flutter bisa "menyuntikkan" nilai konstanta saat build. Nilai itu dibaca dengan `String.fromEnvironment`:

`lib/config/env.dart`

```dart
// lib/config/env.dart
class Env {
  // Nilai diisi saat build oleh --dart-define / --dart-define-from-file.
  static const String supabaseUrl = String.fromEnvironment('SUPABASE_URL');
  static const String supabasePublishableKey = String.fromEnvironment(
    'SUPABASE_PUBLISHABLE_KEY',
  );

  static bool get isConfigured =>
      supabaseUrl.isNotEmpty && supabasePublishableKey.isNotEmpty;
}
```

Jalankan dengan:

```
flutter run \
  --dart-define=SUPABASE_URL=https://abcdefghijklm.supabase.co \
  --dart-define=SUPABASE_PUBLISHABLE_KEY=sb_publishable_xxxxxxxxxxxx
```

- `String.fromEnvironment` **harus** dipakai sebagai `const`. Nilainya ditentukan saat kompilasi, bukan saat aplikasi berjalan.
- Jika lupa memberi `--dart-define`, nilainya string kosong `''`. Karena itu kita sediakan `isConfigured` untuk menampilkan pesan yang jelas.

Cara 2 (disarankan): `--dart-define-from-file`

Mengetik dua define setiap kali menjalankan aplikasi cukup merepotkan. Simpan saja di file JSON di root proyek (sejajar `pubspec.yaml`):

`env.json`

```
{
  "SUPABASE_URL": "https://abcdefghijklm.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_xxxxxxxxxxxxxxxxxxxx"
}
```

```
flutter run --dart-define-from-file=env.json
```

Lalu cegah file itu ikut ter-commit dengan menambahkan baris berikut ke `.gitignore`, dan sediakan contoh tanpa isi untuk teman satu tim:

`.gitignore`

```
# Konfigurasi rahasia/pribadi
env.json
```

`env.example.json`

```
{
  "SUPABASE_URL": "https://YOUR-PROJECT-REF.supabase.co",
  "SUPABASE_PUBLISHABLE_KEY": "sb_publishable_YOUR_KEY"
}
```

Agar tombol Run di VS Code ikut membaca env.json

Buat file `.vscode/launch.json`:

`.vscode/launch.json`

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "catatan_kuliah",
      "request": "launch",
      "type": "dart",
      "args": ["--dart-define-from-file=env.json"]
    }
  ]
}
```

Di Android Studio: *Run* → *Edit Configurations* → isi **Additional run args** dengan `--dart-define-from-file=env.json`.

Inisialisasi memakai Env

```dart
Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Supabase.initialize(
    url: Env.supabaseUrl,
    publishableKey: Env.supabasePublishableKey,
  );

  runApp(const MyApp());
}
```

> ⚠️ **Sudah terlanjur push key ke GitHub?**
>
> Hapus dari repo saja tidak cukup karena riwayat commit masih menyimpannya. Buka **Project Settings → API Keys**, buat kunci baru, lalu hapus (*revoke*) kunci lama. Hal ini **wajib** dilakukan segera jika yang bocor adalah secret key.

5. Membuat Tabel dengan SQL Editor

Tabel bisa dibuat lewat **Table Editor** dengan klik-klik, tetapi di praktikum ini kita memakai **SQL Editor**. Perintah SQL bisa disimpan, dibagikan, dan dijalankan ulang persis sama. Kebiasaan ini juga yang dipakai tim profesional (disebut *migration*).

Membuka SQL Editor

1. Di sidebar dashboard klik **SQL Editor**.
2. Klik **New query** (atau ikon +).
3. Tempel SQL, lalu klik **Run** (atau `Ctrl`/`Cmd` + `Enter`).
4. Hasil atau pesan error muncul di panel bawah. "Success. No rows returned" artinya perintah berhasil.

Tabel pertama: `mata_kuliah`

```
create table public.mata_kuliah (
  id bigint generated always as identity primary key,
  kode text not null unique,
  nama text not null,
  sks int not null default 3 check (sks between 1 and 6),
  semester int not null check (semester between 1 and 8),
  created_at timestamptz not null default now()
);
```

Kita bedah baris demi baris:

- `public.mata_kuliah` : tabel dibuat di schema `public`. Hanya tabel di schema ini yang bisa diakses lewat API dari Flutter.
- `bigint generated always as identity primary key` : id berupa angka yang naik otomatis (1, 2, 3, ...). Kita tidak perlu mengirim id saat insert.
- `text not null unique` : kode wajib diisi dan tidak boleh ada yang sama. Insert kode ganda akan ditolak dengan error `23505`.
- `default 3` : jika kolom tidak dikirim saat insert, nilainya otomatis 3.
- `check (sks between 1 and 6)` : database menolak nilai di luar rentang (error `23514`). Validasi di database adalah lapis pertahanan terakhir, meskipun form Flutter juga sudah memvalidasi.
- `timestamptz not null default now()` : waktu dibuat terisi otomatis dalam UTC dengan zona waktu.

Mengisi data awal

```
insert into public.mata_kuliah (kode, nama, sks, semester) values
  ('INF101', 'Algoritma dan Pemrograman', 4, 1),
  ('INF102', 'Matematika Diskrit', 3, 1),
  ('INF201', 'Struktur Data', 3, 2),
  ('INF202', 'Basis Data', 3, 3),
```

```
  ('INF301', 'Pemrograman Mobile', 3, 5),
  ('INF302', 'Jaringan Komputer', 3, 4),
  ('INF303', 'Kecerdasan Buatan', 3, 5),
  ('INF304', 'Rekayasa Perangkat Lunak', 3, 4);

-- Cek hasilnya
select * from public.mata_kuliah order by semester;
```

Tipe data PostgreSQL yang sering dipakai

Tipe Postgres	Tipe di Dart	Contoh penggunaan
`text`	`String`	nama, judul, isi catatan (tanpa batas panjang)
`int` / `bigint`	`int`	sks, jumlah, id identity
`numeric` / `float8`	`num` / `double`	IPK, harga
`boolean`	`bool`	`is_done`, `is_pinned`
`timestamptz`	`String` ISO-8601 → `DateTime.parse()`	`created_at`
`date`	`String` `'2026-10-28'`	tanggal lahir, deadline
`uuid`	`String`	id user (`auth.users.id`), id catatan
`jsonb`	`Map` / `List`	data fleksibel/pengaturan

Identity (angka) atau UUID?

```
-- Contoh tabel ber-id UUID (akan kita pakai untuk catatan di Modul 14)
create table public.contoh_uuid (
  id uuid primary key default gen_random_uuid(),
  judul text not null
);
```

	`bigint identity`	`uuid default gen_random_uuid()`
Bentuk	1, 2, 3, ...	`3f6c2a9e-8b1d-4c7a-...`
Bisa ditebak?	Ya (id berikutnya mudah ditebak)	Tidak
Bisa dibuat di HP sebelum online?	Tidak	Ya (package `uuid`)
Cocok untuk	Data referensi/master	Data milik user, sinkronisasi offline

Relasi ke tabel user (`auth.users`)

Supabase menyimpan akun hasil register di tabel `auth.users` (schema `auth`, dikelola Supabase, tidak bisa diakses langsung lewat API). Tabel milik kita cukup menyimpan id user sebagai *foreign key*:

```
create table public.contoh_milik_user (
  id uuid primary key default gen_random_uuid(),
  user_id uuid not null default auth.uid()
    references auth.users (id) on delete cascade,
  isi text not null
);
```

- `default auth.uid()`: saat insert dari aplikasi, `user_id` otomatis terisi id user yang sedang login. Aplikasi tidak perlu mengirimnya.
- `on delete cascade`: jika akun user dihapus, semua barisnya ikut terhapus.

> 💡 **Membersihkan tabel latihan**
>
> Tabel `contoh_uuid` dan `contoh_milik_user` hanya untuk latihan. Hapus dengan `drop table public.contoh_uuid, public.contoh_milik_user;`

> ⚠️ **Tabel baru tanpa RLS = terbuka untuk siapa saja**
>
> Tabel yang dibuat lewat SQL tidak otomatis dilindungi. Selama RLS belum aktif, siapa pun yang memegang publishable key bisa membaca *dan mengubah* isinya (dashboard akan memberi label peringatan **UNRESTRICTED**). Sebaliknya, jika RLS aktif tetapi belum ada policy, hasil query dari aplikasi selalu **kosong**. Section berikutnya membahas cara mengatur keduanya dengan benar.

6. Row Level Security (RLS) & Policy

Publishable key ada di dalam aplikasi, sehingga siapa pun yang membongkar APK bisa memakainya untuk memanggil API Supabase. Lalu apa yang mencegah orang iseng membaca catatan milik mahasiswa lain? Jawabannya adalah **Row Level Security (RLS)**.

Konsep: satpam di setiap baris

Bayangkan setiap tabel dijaga seorang satpam. Saat RLS **aktif**, satpam menolak *semua* akses, kecuali yang diizinkan secara eksplisit oleh **policy**. Policy adalah aturan berbentuk kondisi SQL yang dicek untuk *setiap baris*:

```
Request: SELECT * FROM notes   (dikirim oleh user dengan id = A)

 baris  | user_id | policy: auth.uid() = user_id | hasil
--------+---------+------------------------------+---------------
 note 1 |    A    |  A = A  → true               | ✅ dikirim
 note 2 |    B    |  A = B  → false              | ❌ disembunyikan
 note 3 |    A    |  A = A  → true               | ✅ dikirim
```

Aplikasi tidak perlu menulis `.eq('user_id', ...)`. Database sendiri yang memfilter, sehingga tidak bisa diakali dari sisi aplikasi.

Peran (role) dalam request

Role	Kapan	Nilai `auth.uid()`
`anon`	Request dengan publishable key, user **belum login**	`null`
`authenticated`	User **sudah login** (token sesi otomatis dikirim oleh supabase_flutter)	id user tersebut (uuid)
`service_role`	Memakai secret key (khusus server)	Melewati RLS sepenuhnya

Contoh 1: data publik yang boleh dibaca siapa saja

```sql
-- 1. Aktifkan RLS (setelah ini semua akses ditolak)
alter table public.mata_kuliah enable row level security;

-- 2. Izinkan SELECT untuk semua orang
create policy "Semua orang boleh membaca mata kuliah"
on public.mata_kuliah
for select
to anon, authenticated
using (true);

-- 3. Hanya user yang sudah login boleh menambah
```

```
create policy "User login boleh menambah mata kuliah"
on public.mata_kuliah
for insert
to authenticated
with check (true);
```

Tanpa policy UPDATE dan DELETE, tidak ada seorang pun (dari aplikasi) yang bisa mengubah atau menghapus mata kuliah. Itulah yang kita mau untuk data master.

Contoh 2: data milik pribadi (pola yang paling sering dipakai)

```
create policy "User melihat catatannya sendiri"
on public.notes
for select
to authenticated
using ( (select auth.uid()) = user_id );
```

Kita bedah baris per baris:

Baris	Arti
`create policy "User melihat ..."`	Nama policy, bebas tetapi harus unik per tabel. Tulis nama yang mudah dipahami.
`on public.notes`	Tabel yang diatur.
`for select`	Operasi yang diatur: `select`, `insert`, `update`, `delete`, atau `all`.
`to authenticated`	Policy hanya berlaku untuk user yang sudah login. User anon langsung ditolak.
`using ( ... )`	Kondisi untuk baris yang **sudah ada**: baris mana yang boleh dilihat/diubah/dihapus.
`auth.uid()`	Fungsi bawaan Supabase yang mengembalikan id user dari token login pada request ini.
`(select auth.uid())`	Dibungkus `select` agar Postgres cukup menghitungnya sekali per query, bukan sekali per baris. Hasilnya sama tetapi lebih cepat.
`= user_id`	Baris lolos hanya jika kolom `user_id` sama dengan id user yang login.

`using` vs `with check`

Operasi	`using` (baris lama)	`with check` (baris baru)
SELECT	✅ dipakai	–
INSERT	–	✅ dipakai: data yang akan masuk harus memenuhi syarat
UPDATE	✅ baris mana yang boleh diubah	✅ hasil perubahan harus tetap memenuhi syarat
DELETE	✅ dipakai	–

```
create policy "User menambah catatannya sendiri"
on public.notes for insert to authenticated
with check ( (select auth.uid()) = user_id );

create policy "User mengubah catatannya sendiri"
on public.notes for update to authenticated
using ( (select auth.uid()) = user_id )        -- hanya baris miliknya
with check ( (select auth.uid()) = user_id );  -- tidak boleh "memindahkan" ke user lain

create policy "User menghapus catatannya sendiri"
on public.notes for delete to authenticated
using ( (select auth.uid()) = user_id );
```

Apa yang terjadi jika ditolak RLS?

Operasi	Gejala di Flutter
SELECT	**Tidak error**, hanya mengembalikan list kosong `[]`. Ini yang paling sering membingungkan!
INSERT	`PostgrestException` dengan code `42501`: *new row violates row-level security policy*
UPDATE / DELETE	Tidak error, tetapi tidak ada baris yang berubah (0 baris cocok). Jika memakai `.select().single()`, muncul error karena hasilnya 0 baris.

> 💡 **Melihat dan menguji policy**
>
> Buka **Authentication → Policies** (atau Database → Policies) untuk melihat semua policy per tabel. Di SQL Editor, query dijalankan sebagai admin (`postgres`) sehingga RLS *tidak* berlaku. Jadi data yang terlihat di SQL Editor belum tentu terlihat dari aplikasi.

⚠️ **Kesalahan umum RLS**

- **Lupa** `enable row level security` : tabel terbuka untuk publik. Dashboard menandainya *UNRESTRICTED*.
- **RLS aktif tetapi lupa membuat policy SELECT**: data "hilang", list selalu kosong.
- **Policy** `using (true)` **untuk data pribadi**: semua user bisa melihat data user lain.
- **Insert tanpa** `user_id` **dan tanpa** `default auth.uid()` : `user_id` bernilai null sehingga `with check` gagal dan muncul error 42501.

7. Query Dasar: Select, Insert, Update, Delete, Upsert

Setelah tabel dan policy siap, saatnya mengakses data dari Flutter. Semua query dimulai dengan `supabase.from('nama_tabel')`, lalu disambung method lain seperti rantai (*method chaining*), mirip menyusun kalimat SQL.

Semua contoh di section ini mengasumsikan bagian atas file berisi:

```dart
import 'package:flutter/foundation.dart'; // untuk debugPrint
import 'package:supabase_flutter/supabase_flutter.dart';

final supabase = Supabase.instance.client;
```

Pemetaan SQL ke Dart

SQL	supabase_flutter
`SELECT * FROM t`	`.from('t').select()`
`WHERE a = 1`	`.eq('a', 1)`
`WHERE a <> 1`	`.neq('a', 1)`
`>`, `>=`, `<`, `<=`	`.gt()`, `.gte()`, `.lt()`, `.lte()`
`WHERE nama ILIKE '%x%'`	`.ilike('nama', '%x%')`
`WHERE a IN (1,2)`	`.inFilter('a', [1, 2])`
`ORDER BY a DESC`	`.order('a', ascending: false)`
`LIMIT 10`	`.limit(10)`
`LIMIT 10 OFFSET 20`	`.range(20, 29)`
`INSERT` / `UPDATE` / `DELETE`	`.insert({...})` / `.update({...})` / `.delete()`

1. SELECT: membaca data

Pertama, siapkan model seperti di Modul 09 agar data aman dari salah tipe:

```dart
class MataKuliah {
  const MataKuliah({
    required this.id,
    required this.kode,
```

```
    required this.nama,
    required this.sks,
    required this.semester,
  });

  final int id;
  final String kode;
  final String nama;
  final int sks;
  final int semester;

  // Satu baris hasil query Supabase = Map<String, dynamic>
  factory MataKuliah.fromJson(Map<String, dynamic> json) {
    return MataKuliah(
      id: json['id'] as int,
      kode: json['kode'] as String,
      nama: json['nama'] as String,
      sks: json['sks'] as int,
      semester: json['semester'] as int,
    );
  }
}
```

```
Future<List<MataKuliah>> ambilSemua() async {
  // SELECT * FROM mata_kuliah
  final List<Map<String, dynamic>> rows = await supabase
      .from('mata_kuliah')
      .select();
  return rows.map(MataKuliah.fromJson).toList();
}
```

- Hasil `select()` berupa `List<Map<String, dynamic>>`. Satu Map adalah satu baris, dengan key berupa nama kolom.
- `rows.map(MataKuliah.fromJson).toList()` mengubah setiap Map menjadi objek. Polanya sama persis dengan Modul 09.

Mengambil sebagian kolom saja (lebih hemat kuota dan lebih cepat):

```
Future<void> ambilKolomTertentu() async {
  // SELECT kode, nama FROM mata_kuliah
  final rows = await supabase.from('mata_kuliah').select('kode, nama');
  for (final row in rows) {
    debugPrint('${row['kode']} - ${row['nama']}');
  }
}
```

2. Filter

```
Future<List<Map<String, dynamic>>> contohFilter() async {
  // WHERE semester = 5
  final semester5 = await supabase
      .from('mata_kuliah')
      .select()
      .eq('semester', 5);
```

```dart
  // WHERE sks >= 4
  final sksBesar = await supabase.from('mata_kuliah').select().gte('sks', 4);

  // WHERE nama ILIKE '%data%'  (cari teks, tidak peka huruf besar/kecil)
  final cari = await supabase
      .from('mata_kuliah')
      .select()
      .ilike('nama', '%data%');

  // WHERE semester IN (1, 2)
  final awal = await supabase.from('mata_kuliah').select().inFilter(
    'semester',
    [1, 2],
  );

  return [...semester5, ...sksBesar, ...cari, ...awal];
}
```

3. Urutan dan batas jumlah

```dart
Future<List<Map<String, dynamic>>> urutDanBatasi() async {
  // SELECT * FROM mata_kuliah WHERE semester <= 4
  // ORDER BY semester ASC, nama ASC LIMIT 5
  return supabase
      .from('mata_kuliah')
      .select()
      .lte('semester', 4) // filter dulu
      .order('semester', ascending: true) // lalu urutkan
      .order('nama', ascending: true)
      .limit(5); // terakhir batasi jumlah
}
```

> **Urutan method**
>
> Tulis **filter** (`eq`, `gte`, ...) lebih dulu, baru `order`, `limit` / `range`, dan `single`. Setelah `.order()`, method filter sudah tidak tersedia, dan analyzer akan langsung memberi tahu jika urutannya salah.

4. Pagination dengan `range`

```dart
/// Pagination: halaman dimulai dari 0, setiap halaman berisi [perHalaman] baris.
Future<List<Map<String, dynamic>>> ambilHalaman(
  int halaman, {
  int perHalaman = 3,
}) {
  final dari = halaman * perHalaman; // halaman 0 -> 0, halaman 1 -> 3
  final sampai = dari + perHalaman - 1; // halaman 0 -> 2, halaman 1 -> 5
  return supabase
      .from('mata_kuliah')
```

```
      .select()
      .order('id')
      .range(dari, sampai); // indeks inklusif di kedua sisi
}
```

`range(dari, sampai)` memakai indeks mulai 0 dan **inklusif** di kedua ujung. `range(0, 2)` berarti 3 baris pertama. Panggil `ambilHalaman(0)`, `ambilHalaman(1)`, dan seterusnya saat user menggulir ke bawah (*infinite scroll*, Modul 06).

5. Mengambil satu baris: `single()` dan `maybeSingle()`

```
Future<void> ambilSatu() async {
  // single(): WAJIB tepat 1 baris. 0 atau >1 baris -> PostgrestException.
  final Map<String, dynamic> row = await supabase
      .from('mata_kuliah')
      .select()
      .eq('kode', 'INF301')
      .single();
  debugPrint(row['nama']);

  // maybeSingle(): 0 baris -> null (tidak error).
  final Map<String, dynamic>? mungkin = await supabase
      .from('mata_kuliah')
      .select()
      .eq('kode', 'TIDAK-ADA')
      .maybeSingle();
  if (mungkin == null) debugPrint('Mata kuliah tidak ditemukan');
}
```

Method	Tipe hasil	0 baris	1 baris	>1 baris
(tanpa)	`List<Map>`	`[]`	`[{...}]`	`[{...}, ...]`
`.single()`	`Map`	❌ error	✅ Map	❌ error
`.maybeSingle()`	`Map?`	✅ `null`	✅ Map	❌ error

6. INSERT: menambah data

```
Future<void> tambah() async {
  // Insert tanpa meminta data balik
  await supabase.from('mata_kuliah').insert({
    'kode': 'INF401',
    'nama': 'Komputasi Awan',
    'sks': 3,
    'semester': 7,
  });

  // Insert banyak baris sekaligus: kirim List<Map>
  await supabase.from('mata_kuliah').insert([
    {'kode': 'INF402', 'nama': 'Data Mining', 'semester': 7},
```

```
    {'kode': 'INF403', 'nama': 'Etika Profesi', 'sks': 2, 'semester': 8},
  ]);
}

Future<MataKuliah> tambahDanAmbil() async {
  // .select().single() -> baris yang baru dibuat dikembalikan,
  // lengkap dengan id dan created_at yang diisi database.
  final row = await supabase
      .from('mata_kuliah')
      .insert({'kode': 'INF404', 'nama': 'Keamanan Siber', 'semester': 7})
      .select()
      .single();
  return MataKuliah.fromJson(row);
}
```

- Kolom yang punya `default` (id, sks, created_at) boleh tidak dikirim.
- Secara default insert **tidak** mengembalikan data. Tambahkan `.select()` (dan `.single()` untuk satu baris) jika butuh id yang dibuat database. Pola ini akan kita pakai di Modul 14.

7. UPDATE: mengubah data

```
Future<void> ubah() async {
  // UPDATE mata_kuliah SET sks = 4 WHERE kode = 'INF401'
  await supabase
      .from('mata_kuliah')
      .update({'sks': 4})
      .eq('kode', 'INF401'); // JANGAN lupa filter!
}
```

> ⚠️ **Selalu pakai filter**
>
> `update()` dan `delete()` **wajib** diikuti filter seperti `.eq('id', ...)`. Di project Supabase, update/delete tanpa filter biasanya ditolak dengan pesan *UPDATE/DELETE requires a WHERE clause*. Filter yang salah lebih berbahaya lagi karena bisa mengubah baris yang tidak Anda maksud.

8. DELETE: menghapus data

```
Future<void> hapus() async {
  // DELETE FROM mata_kuliah WHERE kode = 'INF401'
  await supabase.from('mata_kuliah').delete().eq('kode', 'INF401');
}
```

Catatan: dengan policy dari section sebelumnya, tabel `mata_kuliah` tidak punya policy UPDATE/DELETE, sehingga dua contoh di atas tidak mengubah apa pun jika dijalankan dari aplikasi (0 baris). Ini contoh nyata RLS bekerja.

9. UPSERT: insert atau update

```dart
Future<void> simpanAtauPerbarui() async {
  // Jika kode 'INF301' sudah ada -> UPDATE, jika belum -> INSERT.
  // onConflict menunjuk kolom UNIQUE yang dipakai untuk mencocokkan.
  await supabase.from('mata_kuliah').upsert({
    'kode': 'INF301',
    'nama': 'Pemrograman Mobile',
    'sks': 4,
    'semester': 5,
  }, onConflict: 'kode');
}
```

Upsert berguna untuk sinkronisasi, misalnya "simpan profil": jika belum ada dibuat, jika sudah ada diperbarui. Ingat, upsert butuh policy **INSERT dan UPDATE** sekaligus.

10. Menghitung jumlah baris

```dart
Future<int> hitung() async {
  // Hanya menghitung jumlah baris tanpa mengambil datanya.
  return supabase.from('mata_kuliah').count();
}
```

Contoh lengkap: daftar mata kuliah dari Supabase

Gabungkan semuanya menjadi aplikasi kecil yang bisa dijalankan. Pastikan SQL di section 5 dan 6 sudah dijalankan, dan `env.json` sudah dibuat (section 4).

`lib/main.dart`

```dart
// lib/main.dart — contoh lengkap: membaca tabel mata_kuliah dari Supabase.
// Jalankan: flutter run --dart-define-from-file=env.json
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

const supabaseUrl = String.fromEnvironment('SUPABASE_URL');
const supabaseKey = String.fromEnvironment('SUPABASE_PUBLISHABLE_KEY');

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Supabase.initialize(url: supabaseUrl, publishableKey: supabaseKey);
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
      ),
      home: const MataKuliahPage(),
```

```
    );
  }
}

class MataKuliah {
  const MataKuliah({
    required this.kode,
    required this.nama,
    required this.sks,
    required this.semester,
  });

  final String kode;
  final String nama;
  final int sks;
  final int semester;

  factory MataKuliah.fromJson(Map<String, dynamic> json) {
    return MataKuliah(
      kode: json['kode'] as String,
      nama: json['nama'] as String,
      sks: json['sks'] as int,
      semester: json['semester'] as int,
    );
  }
}

class MataKuliahPage extends StatefulWidget {
  const MataKuliahPage({super.key});

  @override
  State<MataKuliahPage> createState() => _MataKuliahPageState();
}

class _MataKuliahPageState extends State<MataKuliahPage> {
  final _supabase = Supabase.instance.client;
  late Future<List<MataKuliah>> _future;

  @override
  void initState() {
    super.initState();
    _future = _ambilData();
  }

  Future<List<MataKuliah>> _ambilData() async {
    final rows = await _supabase
        .from('mata_kuliah')
        .select()
        .order('semester')
        .order('nama');
    return rows.map(MataKuliah.fromJson).toList();
  }

  Future<void> _refresh() async {
    final future = _ambilData();
    setState(() => _future = future);
    await future;
  }
```

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Mata Kuliah Informatika')),
      body: FutureBuilder<List<MataKuliah>>(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.connectionState == ConnectionState.waiting) {
            return const Center(child: CircularProgressIndicator());
          }
          if (snapshot.hasError) {
            return Center(
              child: Padding(
                padding: const EdgeInsets.all(24),
                child: Column(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    Text(
                      'Gagal memuat data:\n${snapshot.error}',
                      textAlign: TextAlign.center,
                    ),
                    const SizedBox(height: 12),
                    FilledButton(
                      onPressed: _refresh,
                      child: const Text('Coba lagi'),
                    ),
                  ],
                ),
              ),
            );
          }
          final data = snapshot.data!;
          if (data.isEmpty) {
            return const Center(child: Text('Tabel masih kosong.'));
          }
          return RefreshIndicator(
            onRefresh: _refresh,
            child: ListView.separated(
              itemCount: data.length,
              separatorBuilder: (_, _) => const Divider(height: 1),
              itemBuilder: (context, index) {
                final mk = data[index];
                return ListTile(
                  leading: CircleAvatar(child: Text('${mk.semester}')),
                  title: Text(mk.nama),
                  subtitle: Text(mk.kode),
                  trailing: Text('${mk.sks} SKS'),
                );
              },
            ),
          );
        },
      ),
    );
  }
}
```



Data tabel mata_kuliah diambil dari project Supabase sungguhan

- `_future` disimpan di State dan dibuat di `initState`, **bukan** di `build()`. Jika dibuat di build, setiap rebuild akan memicu query baru (pelajaran yang sama dengan FutureBuilder di Modul 09).
- `_refresh()` membuat Future baru dan memasangnya lewat `setState`, lalu menunggu sampai selesai agar animasi `RefreshIndicator` berhenti tepat waktu.
- `.order('semester').order('nama')` mengurutkan per semester, lalu per nama.

8. Menangani Error Supabase

Request ke internet *pasti* suatu saat gagal, entah karena sinyal hilang, server menolak, atau data tidak valid. Aplikasi yang baik menampilkan pesan yang jelas, bukan layar merah atau loading tanpa akhir.

Tiga jenis kegagalan

Jenis	Exception di Dart	Contoh penyebab
Database menolak	`PostgrestException`	RLS, UNIQUE dilanggar, CHECK gagal, tabel tidak ada
Auth menolak	`AuthException` (dan turunannya)	password salah, email belum dikonfirmasi, email sudah terdaftar
Jaringan gagal	`SocketException`, `ClientException`, `TimeoutException`, `AuthRetryableFetchException`	tidak ada internet, DNS gagal, server terlalu lama

Menangani `PostgrestException`

```
Future<String> tambahDenganPenangananError() async {
  try {
    await supabase.from('mata_kuliah').insert({
      'kode': 'INF301', // sudah ada -> melanggar UNIQUE
      'nama': 'Duplikat',
      'semester': 5,
    });
    return 'Berhasil disimpan';
  } on PostgrestException catch (e) {
    // e.code berisi kode error PostgreSQL / PostgREST
    switch (e.code) {
      case '23505':
        return 'Kode mata kuliah sudah dipakai.';
      case '23514':
        return 'Data melanggar aturan CHECK (misalnya SKS di luar 1-6).';
      case '42501':
        return 'Ditolak oleh Row Level Security. Apakah Anda sudah login?';
      default:
        return 'Kesalahan database: ${e.message}';
    }
  } on TimeoutException {
    return 'Server terlalu lama merespons.';
  } catch (e) {
    // Biasanya SocketException / ClientException: tidak ada internet.
    return 'Tidak dapat terhubung ke server.';
```

```
  }
}
```

- `on PostgrestException catch (e)` hanya menangkap error dari database. Properti pentingnya: `e.message`, `e.code`, `e.details`, dan `e.hint`.
- `catch (e)` paling bawah menangkap sisanya, biasanya masalah jaringan.
- Urutan `on ... catch` penting: yang paling spesifik ditulis di atas.

Kode error yang sering muncul

Code	Arti	Solusi
`42501`	Ditolak RLS (*new row violates row-level security policy*)	Periksa policy INSERT/UPDATE dan pastikan user sudah login.
`23505`	Nilai UNIQUE ganda (*duplicate key value*)	Beri tahu user bahwa data sudah ada.
`23502`	Kolom NOT NULL tidak diisi	Lengkapi data yang dikirim.
`23503`	Foreign key tidak ditemukan	Pastikan baris induk ada.
`23514`	Constraint CHECK gagal	Validasi input di form.
`22P02`	Format salah, misalnya teks dikirim ke kolom uuid/int	Periksa tipe data.
`PGRST116`	`.single()` menerima 0 atau lebih dari 1 baris	Pakai `maybeSingle()` atau perbaiki filter.
`PGRST205` / `42P01`	Tabel tidak ditemukan	Jalankan SQL pembuatan tabel, cek ejaan nama tabel.

Menangani `AuthException`

```
Future<String?> loginDenganPenangananError(
  String email,
  String password,
) async {
  try {
    await supabase.auth.signInWithPassword(email: email, password: password);
    return null; // null = sukses
  } on AuthRetryableFetchException {
    return 'Tidak ada koneksi internet.';
  } on AuthException catch (e) {
    // e.code: 'invalid_credentials', 'email_not_confirmed', dst.
```

```
      if (e.code == 'invalid_credentials') return 'Email atau password salah.';
      if (e.code == 'email_not_confirmed') return 'Konfirmasi email dulu.';
      return e.message;
    }
  }
```

`AuthRetryableFetchException` adalah turunan `AuthException` yang dilempar saat request Auth gagal karena jaringan. Karena merupakan turunan, ia harus ditangkap **sebelum** `on AuthException`.

> 💡 **Di mana sebaiknya try-catch ditulis?**
>
> Jangan menyebar try-catch di setiap widget. Di proyek Modul 13–14, error ditangkap di **Provider**, diterjemahkan menjadi pesan Bahasa Indonesia, lalu UI cukup menampilkan pesannya. Dengan begitu tampilan tetap bersih dan logika error terkumpul di satu tempat.

> ⚠️ **Jangan menelan error**
>
> `catch (e) {}` yang kosong membuat bug tidak terlihat: data tidak tersimpan dan user tidak tahu kenapa. Minimal tampilkan SnackBar atau tulis `debugPrint(e.toString())` saat pengembangan.

9. Dasar Supabase Auth

Supabase Auth mengurus register, login, penyimpanan sesi, dan perpanjangan token secara otomatis. Di section ini kita mengenal method-method dasarnya. Implementasi lengkap dengan UI dikerjakan di Modul 13.

Pengaturan di dashboard

1. Buka **Authentication → Sign In / Providers**. Pastikan provider **Email** aktif (default sudah aktif).
2. Perhatikan opsi **Confirm email**:
   - **ON** (default): setelah register, user harus mengeklik link di email sebelum bisa login. `signUp` mengembalikan `session == null`.
   - **OFF**: user langsung login setelah register. `signUp` mengembalikan session yang valid. Cocok untuk latihan di kelas.
3. Daftar user yang sudah register bisa dilihat di **Authentication → Users**.

> ⚠️ **Batas pengiriman email**
>
> Layanan email bawaan Supabase hanya untuk percobaan dan dibatasi beberapa email per jam. Jika satu kelas register bersamaan dengan Confirm email ON, sebagian akan mendapat error `over_email_send_rate_limit`. Untuk praktikum, matikan Confirm email. Untuk aplikasi produksi, pasang SMTP sendiri di **Authentication → Emails → SMTP Settings**.

1. Register: `signUp`

```dart
Future<void> daftar() async {
  final AuthResponse res = await supabase.auth.signUp(
    email: 'budi@mhs.unsoed.ac.id',
    password: 'rahasia123',
    data: {'full_name': 'Budi Santoso'}, // disimpan di user metadata
  );

  final User? user = res.user; // akun yang baru dibuat
  final Session? session = res.session; // null jika wajib konfirmasi email
  debugPrint('User: ${user?.id}, langsung login: ${session != null}');
}
```

- `data` disimpan sebagai **user metadata** (di database bernama kolom `raw_user_meta_data`). Di Modul 13 kita memakai trigger untuk menyalin `full_name` ke tabel `profiles`.

- Password minimal 6 karakter (default Supabase). Jika kurang, muncul `AuthException` dengan code `weak_password`.

2. Login dan logout

```dart
Future<void> masuk() async {
  final res = await supabase.auth.signInWithPassword(
    email: 'budi@mhs.unsoed.ac.id',
    password: 'rahasia123',
  );
  debugPrint('Halo ${res.user?.email}');
}

Future<void> keluar() async {
  await supabase.auth.signOut();
}
```

Setelah login berhasil, **semua** query database berikutnya otomatis membawa token user. RLS pun mengenali siapa yang meminta data melalui `auth.uid()`, tanpa kode tambahan.

3. Mengecek sesi saat ini

```dart
void cekSesi() {
  // Sesi disimpan otomatis di perangkat oleh supabase_flutter,
  // jadi setelah aplikasi ditutup dan dibuka lagi user tetap login.
  final Session? session = supabase.auth.currentSession;
  final User? user = supabase.auth.currentUser;

  if (session == null) {
    debugPrint('Belum login');
  } else {
    debugPrint('Login sebagai ${user!.email}');
    debugPrint('Nama: ${user.userMetadata?['full_name']}');
  }
}
```

4. Mendengarkan perubahan status login

```dart
StreamSubscription<AuthState> dengarkanAuth() {
  return supabase.auth.onAuthStateChange.listen((AuthState data) {
    final AuthChangeEvent event = data.event;
    final Session? session = data.session;

    switch (event) {
      case AuthChangeEvent.initialSession:
        debugPrint('Aplikasi dibuka, sesi awal: ${session != null}');
      case AuthChangeEvent.signedIn:
        debugPrint('Berhasil login');
      case AuthChangeEvent.signedOut:
        debugPrint('Sudah logout');
      case AuthChangeEvent.tokenRefreshed:
        debugPrint('Token diperbarui otomatis');
      default:
        debugPrint('Event lain: ${event.name}');
```

```
    }
  });
}
// Jangan lupa: subscription.cancel() di dispose() jika dipakai di widget.
```

Event	Kapan terjadi
`initialSession`	Selalu menjadi event pertama saat stream didengarkan. Berisi sesi tersimpan (atau null).
`signedIn`	Setelah login/register berhasil (dan sesi aktif).
`signedOut`	Setelah `signOut()` atau sesi tidak valid lagi.
`tokenRefreshed`	Token akses diperpanjang otomatis (kira-kira setiap jam).
`userUpdated`	Data user diubah lewat `updateUser()`.
`passwordRecovery`	User membuka link reset password.

Stream inilah kunci navigasi otomatis: **ada sesi = tampilkan Home, tidak ada sesi = tampilkan Login**. Pola ini disebut *AuthGate* dan dibangun di Modul 13.

> ℹ️ **Di mana sesi disimpan?**
>
> `supabase_flutter` menyimpan sesi di penyimpanan lokal perangkat (berbasis SharedPreferences, Modul 10). Karena itu user tetap login walaupun aplikasi ditutup, dan token diperbarui otomatis selama aplikasi berjalan.

> ⚠️ **Register dengan email yang sudah terdaftar**
>
> Jika Confirm email ON, Supabase **tidak** melempar error saat email sudah terdaftar. Ia mengembalikan user "palsu" agar penyerang tidak bisa menebak email siapa saja yang terdaftar (*user enumeration*). Jika Confirm email OFF, error `user_already_exists` akan muncul.

10. Sekilas Realtime & Storage

Dua fitur tambahan ini tidak wajib untuk proyek Modul 13–14, tetapi sering diminta di tugas besar: data yang berubah *live* (Realtime) dan unggah file (Storage).

Realtime: data yang otomatis ter-update

Langkah 1: aktifkan Realtime untuk tabel. Caranya bisa lewat **Table Editor** → pilih tabel → aktifkan **Enable Realtime**, atau lewat SQL:

```
alter publication supabase_realtime add table public.mata_kuliah;
```

Langkah 2: ganti `select()` dengan `stream(primaryKey: [...])`, dan `FutureBuilder` dengan `StreamBuilder`:

`lib/main.dart`

```
// lib/main.dart — contoh Realtime: daftar mata kuliah yang otomatis
// berubah ketika tabel diubah dari dashboard atau perangkat lain.
// Jalankan: flutter run --dart-define-from-file=env.json
import 'package:flutter/material.dart';
import 'package:supabase_flutter/supabase_flutter.dart';

const supabaseUrl = String.fromEnvironment('SUPABASE_URL');
const supabaseKey = String.fromEnvironment('SUPABASE_PUBLISHABLE_KEY');

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Supabase.initialize(url: supabaseUrl, publishableKey: supabaseKey);
  runApp(
    const MaterialApp(debugShowCheckedModeBanner: false, home: RealtimePage()),
  );
}

class RealtimePage extends StatefulWidget {
  const RealtimePage({super.key});

  @override
  State<RealtimePage> createState() => _RealtimePageState();
}

class _RealtimePageState extends State<RealtimePage> {
  // Stream disimpan di State: JANGAN dibuat di dalam build().
  final Stream<List<Map<String, dynamic>>> _stream = Supabase.instance.client
      .from('mata_kuliah')
      .stream(primaryKey: ['id'])
      .order('kode');

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Mata Kuliah (Realtime)')),
      body: StreamBuilder<List<Map<String, dynamic>>>(
        stream: _stream,
```

```
        builder: (context, snapshot) {
          if (snapshot.hasError) {
            return Center(child: Text('Error: ${snapshot.error}'));
          }
          if (!snapshot.hasData) {
            return const Center(child: CircularProgressIndicator());
          }
          final rows = snapshot.data!;
          return ListView.builder(
            itemCount: rows.length,
            itemBuilder: (context, index) {
              final row = rows[index];
              return ListTile(
                leading: const Icon(Icons.bolt),
                title: Text(row['nama'] as String),
                subtitle: Text('${row['kode']} • semester ${row['semester']}'),
              );
            },
          );
        },
      ),
    );
  }
}
```



StreamBuilder + stream(): data awal dari Supabase, lalu ikut berubah saat tabel diubah

- `stream(primaryKey: ['id'])` mula-mula memancarkan seluruh data, lalu memancarkan list baru setiap kali ada INSERT/UPDATE/DELETE. `primaryKey` dipakai untuk mencocokkan baris yang berubah.

- Stream disimpan sebagai field State. Membuatnya di dalam `build()` akan membuka koneksi baru di setiap rebuild.
- RLS tetap berlaku: user hanya menerima perubahan baris yang boleh ia lihat.
- Coba buka Table Editor, ubah nama satu mata kuliah, lalu lihat aplikasi berubah sendiri tanpa refresh.

> ℹ️ **Kapan memakai realtime?**
>
> Chat, papan skor, status pesanan, kolaborasi. Untuk daftar yang jarang berubah (seperti catatan pribadi), `select()` biasa plus pull-to-refresh sudah cukup dan lebih hemat kuota koneksi.

Storage: menyimpan file

1. Buka **Storage** → **New bucket**, beri nama `avatars`. Centang **Public bucket** agar file bisa dibuka lewat URL.
2. Storage juga dilindungi RLS. Buat policy agar user hanya boleh mengunggah ke foldernya sendiri:

```
create policy "User mengunggah ke foldernya sendiri"
on storage.objects for insert to authenticated
with check (
  bucket_id = 'avatars'
  and (storage.foldername(name))[1] = (select auth.uid())::text
);

create policy "User menimpa file di foldernya sendiri"
on storage.objects for update to authenticated
using (
  bucket_id = 'avatars'
  and (storage.foldername(name))[1] = (select auth.uid())::text
);
```

Kode unggah (byte gambar bisa didapat dari package seperti `image_picker` dengan `await file.readAsBytes()`):

```
Future<String> unggahFoto(Uint8List bytes) async {
  final userId = supabase.auth.currentUser!.id;
  // Simpan di folder milik user: avatars/<userId>/foto.jpg
  final path = '$userId/foto.jpg';

  await supabase.storage
      .from('avatars')
      .uploadBinary(
        path,
        bytes,
        fileOptions: const FileOptions(
```

```
          contentType: 'image/jpeg',
          upsert: true, // timpa jika sudah ada
        ),
      );

  // URL publik (bucket harus bertipe Public)
  return supabase.storage.from('avatars').getPublicUrl(path);
}
```

- `uploadBinary` menerima `Uint8List` sehingga bisa dipakai di Android, iOS, maupun Web.
- `storage.foldername(name)[1]` di policy adalah nama folder pertama dari path file, yang harus sama dengan id user.
- `upsert: true` membutuhkan policy UPDATE juga, karena file lama ditimpa.
- Untuk bucket privat, gunakan `createSignedUrl(path, detik)` sebagai pengganti `getPublicUrl`.

Ringkasan Modul 12

- Supabase = PostgreSQL + Auth + RLS + Realtime + Storage, diakses lewat `supabase_flutter`.
- `Supabase.initialize(url:, publishableKey:)` sekali di `main()`. Simpan nilai di `env.json` + `--dart-define-from-file`.
- Secret/service_role key **tidak pernah** masuk ke aplikasi. Data dilindungi oleh **RLS**.
- Query: `from().select()/insert()/update()/delete()/upsert()` plus filter, order, range, single/maybeSingle.
- Error database = `PostgrestException`, error login = `AuthException`, sisanya biasanya jaringan.

Tugas Praktikum: Praktikum 12: Katalog Mata Kuliah di Supabase

Sasaran Praktikum

Membangun aplikasi Flutter yang membaca dan menambah data mata kuliah di Supabase dengan konfigurasi kunci yang aman dan RLS yang benar.

Langkah Pengerjaan

1. Buat project Supabase baru (region Singapore). Catat Project URL dan publishable key ke file `env.json`, lalu tambahkan `env.json` ke `.gitignore` dan buat `env.example.json`.
2. Jalankan SQL pembuatan tabel `mata_kuliah` beserta data awal minimal 8 baris (section 5).
3. Aktifkan RLS dan buat policy: SELECT untuk `anon, authenticated`, INSERT hanya untuk `authenticated` (section 6).
4. Pasang `supabase_flutter` dengan `flutter pub add supabase_flutter`, inisialisasi memakai `Env` dan jalankan dengan `--dart-define-from-file=env.json`.
5. Tampilkan daftar mata kuliah urut per semester dengan FutureBuilder + RefreshIndicator (contoh lengkap section 7).
6. Tambahkan ChoiceChip filter semester (1–8) yang memakai `.eq('semester', n)` saat dipilih.
7. Tambahkan tombol 'Muat lebih banyak' yang memakai `.range()` dengan 3 data per halaman.
8. Tangani error: matikan internet lalu refresh, dan pastikan pesan Bahasa Indonesia muncul (bukan layar merah).
9. Bonus: tambah form insert mata kuliah baru. Uji saat belum login (harus gagal 42501) dan tampilkan pesan yang sesuai.

Output yang Diharapkan

Aplikasi menampilkan data mata kuliah dari Supabase, bisa difilter per semester dan dipaginasi, menampilkan pesan error yang jelas saat offline, dan repository Git tidak memuat env.json.

Rubrik Penilaian

Setup project, env.json & .gitignore (15%), SQL tabel + constraint benar (15%), RLS & policy tepat (20%), Query select/filter/order/range berjalan (25%), Penanganan loading/error/empty state (15%), Kerapian kode & model fromJson (10%). Bonus insert dengan penanganan 42501: +10%.

Kuis Pemahaman

1. Kunci Supabase manakah yang BOLEH dipasang di dalam aplikasi Flutter?

A. Secret key (sb_secret_...)

B. service_role key

C. Publishable key (sb_publishable_...) atau anon key lama

D. Password database Postgres

2. RLS pada tabel notes sudah aktif tetapi belum ada satu pun policy. Apa yang terjadi saat aplikasi menjalankan supabase.from('notes').select()?

A. Muncul error 404

B. Semua baris dikembalikan

C. Hasilnya list kosong [] tanpa error

D. Aplikasi crash

3. Apa fungsi auth.uid() di dalam policy RLS?

A. Membuat UUID acak baru

B. Mengembalikan id user yang sedang login berdasarkan token pada request

C. Mengembalikan id baris yang sedang diproses

D. Mengembalikan email user

4. Pada policy UPDATE, apa beda klausa using dan with check?

A. Keduanya sama saja

B. using mengecek baris lama yang boleh diubah, with check mengecek data hasil perubahan

C. using untuk insert, with check untuk delete

D. with check hanya dipakai di SELECT

5. Query mana yang mengembalikan null (bukan error) jika tidak ada baris yang cocok?

A. .select().single()

B. .select().maybeSingle()

C. .select().limit(1)

D. .select().range(0, 0)

6. Untuk menampilkan halaman ke-2 (indeks halaman 1) dengan 10 data per halaman, pemanggilan range yang benar adalah...

A. .range(10, 20)

B. .range(1, 10)

C. .range(10, 19)

D. .range(11, 20)

7. Insert ke tabel notes menghasilkan PostgrestException code 42501 'new row violates row-level security policy'. Penyebab paling mungkin adalah...

A. Internet mati

B. User belum login atau user_id yang dikirim tidak sama dengan auth.uid() sehingga with check gagal

C. Nama kolom salah ketik

D. Supabase.initialize belum dipanggil

8. Dengan pengaturan 'Confirm email' AKTIF di dashboard, apa hasil signUp yang berhasil?

A. AuthResponse dengan session terisi sehingga user langsung login

B. AuthResponse dengan user terisi tetapi session null sampai user mengeklik link konfirmasi

C. Selalu melempar AuthException

D. User otomatis terhapus setelah 1 jam

Kunci Jawaban & Pembahasan

1. C — Publishable key (atau anon key versi lama) memang dirancang untuk aplikasi klien. Data tetap aman karena dilindungi RLS. Secret/service_role key melewati RLS sehingga tidak boleh ada di aplikasi.

2. C — Saat RLS aktif, akses ditolak secara default. Untuk SELECT, penolakan berupa 'tidak ada baris yang lolos', sehingga hasilnya list kosong tanpa exception.

3. B — auth.uid() membaca token sesi yang dikirim supabase_flutter dan mengembalikan id user tersebut (null untuk anon). Policy seperti (select auth.uid()) = user_id membatasi akses hanya ke baris miliknya.

4. B — using memfilter baris yang sudah ada (mana yang boleh disentuh), sedangkan with check memvalidasi baris baru/hasil update, misalnya agar user_id tidak bisa diganti ke milik user lain.

5. B — maybeSingle() mengembalikan Map? (null jika 0 baris). single() melempar PostgrestException bila jumlah baris bukan tepat 1, sedangkan limit/range mengembalikan list.

6. C — range memakai indeks mulai 0 dan inklusif di kedua ujung. Halaman 1 dimulai dari 1 × 10 = 10 sampai 10 + 10 - 1 = 19.

7. B — 42501 berarti ditolak oleh policy. Pada insert, with check ((select auth.uid()) = user_id) gagal jika user anon (auth.uid() null) atau user_id berisi id lain.

8. B — Saat konfirmasi email wajib, akun dibuat tetapi belum ada sesi. Aplikasi harus memberi tahu user untuk mengecek email, lalu login setelah konfirmasi.